TradesOn Backend Technical Guide

Architecture, Schema, CRUD Events, Pub/Sub, and Security Posture

TradesOn LLC — Engineering

May 2026

1. Executive Summary

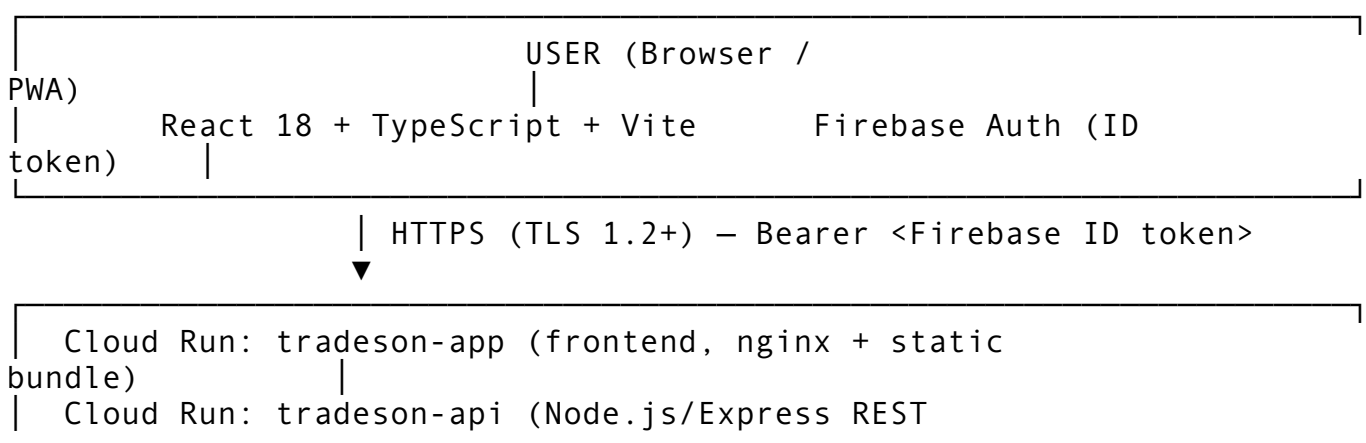
TradesOn is a two-sided home-services marketplace connecting homeowners, property managers, and realtors with verified tradespeople for repairs and maintenance. The platform is built as a SaaS web application (mobile-first responsive) hosted entirely on **Google Cloud Platform**.

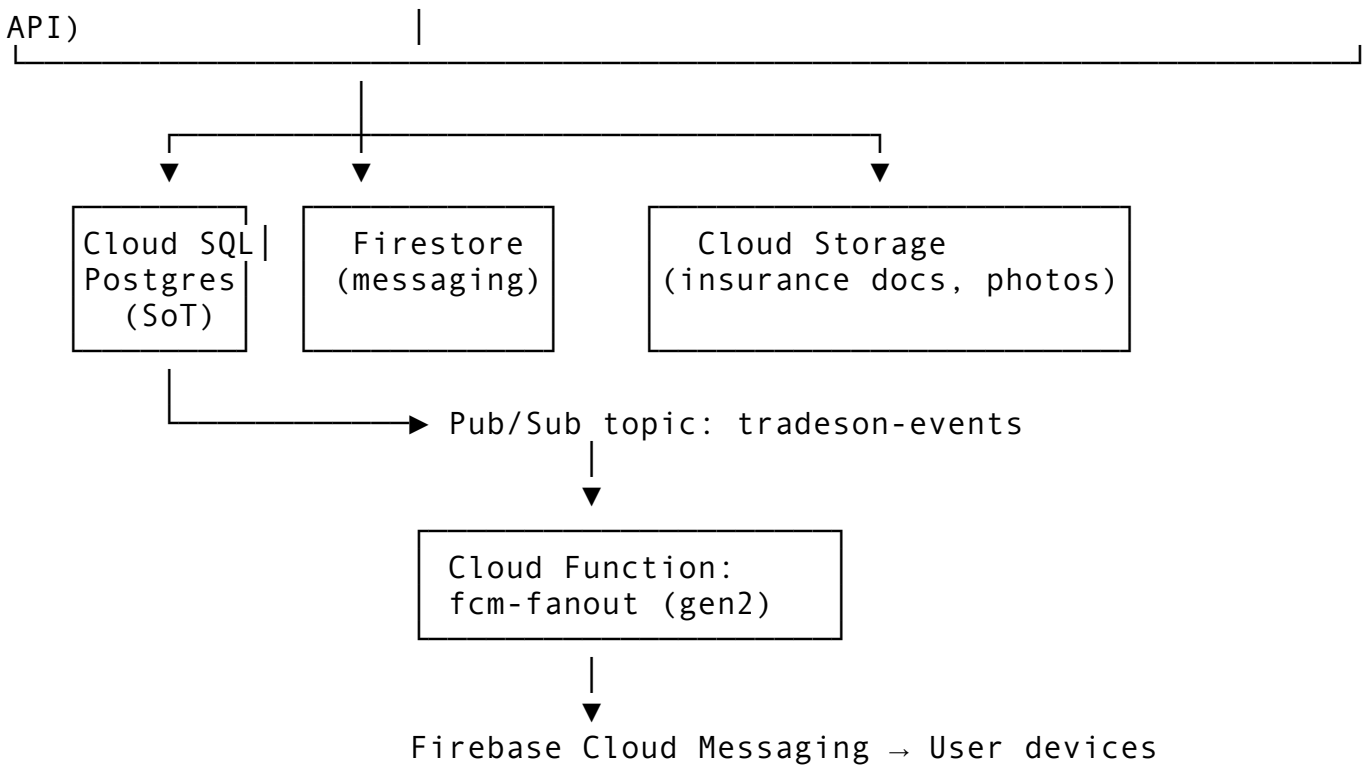
The backend is a single REST API service on **Cloud Run** (containerized Node.js/Express) backed by **Cloud SQL Postgres 16** as the source-of-truth transactional database. **Firestore** holds only real-time messaging. Card payments are processed exclusively through **Stripe Connect** — TradesOn does not store, transmit, or process credit card data, putting the platform out of PCI-DSS scope.

Real-time UX (push notifications for new bids, quote acceptance, schedule changes) is delivered via **Firebase Cloud Messaging (FCM)**, with **Cloud Pub/Sub** acting as the asynchronous decoupling layer between the API and the FCM fan-out worker.

---

2. High-Level Architecture





Three-store data split

Store	What lives there	Why
Cloud SQL Postgres	Users, jobs, quotes, payments, reviews, compliance, audit	Transactional, relational, strong consistency
Firestore	Real-time messaging threads + messages only	Sub-second `onSnapshot` for chat
BigQuery *(planned)*	Analytics, funnel metrics	Decouples analytics reads from the hot path

Money never touches our database. **Stripe Connect Express** holds custody of all funds; we only store opaque references (`stripe_payment_intent_id`, `stripe_account_id`).

---

3. Cloud Pub/Sub — What It Is and Why We Use It

3.1 What Pub/Sub does

Cloud Pub/Sub is a fully managed message-broker service. The Cloud Run API publishes a small JSON event after every meaningful database write. Subscribers (currently one Cloud Function) consume those events asynchronously.

This decouples the user-facing request from any side effect. When a tradesperson submits a quote:

1. Express route writes to Postgres → 201 returned to the client in ~80 ms.

2. The route publishes a `quote.submitted` event to the `tradeson-events` topic. The publish is **fire-and-forget** — the user's response is not gated on it.
3. The `fcm-fanout` Cloud Function consumes the event, looks up the customer's FCM token in Firestore, and pushes a notification.

If FCM is down, the user's quote still saves. If Firestore is slow, the user's request still completes. Pub/Sub absorbs back-pressure and retries failed deliveries automatically (with exponential backoff and a 7-day retention window).

3.2 Event payload contract

Every published message has this shape (defined in `api/src/services/pubsub.ts`):

```
interface FcmEvent {
  event: 'job.created' | 'job.status_changed'
       | 'quote.submitted' | 'quote.accepted'
       | 'message.sent';
  targetUserId: string;       // Firebase UID of the recipient
  title: string;              // notification title
  body: string;               // notification body
  data?: Record<string, string>;  // optional deep-link metadata
}
```

The publish helper is best-effort and never throws — a failed publish is logged but does not break the user-facing API call.

3.3 The fan-out function

`functions/fcm-fanout/index.js` is a Cloud Function (gen2) subscribed to the `tradeson-events` topic. For each event:

1. Decode the Pub/Sub message.
2. Read `users/{targetUserId}.fcmToken` from Firestore.
3. If a token exists, send a notification via `admin.messaging().send()`.
4. If FCM returns `messaging/registration-token-not-registered`, delete the stale token from Firestore.
5. If no token exists (user hasn't logged in on a device that registered), log and exit.

Verified end-to-end on **2026-05-10 17:03 UTC**: 4 test events published from `scripts/testPubsub.mjs` → all 4 consumed by `fcm-fanout` within 1 second.

3.4 Why this matters for security and reliability

- **No SSRF or injection surface added** — the publisher only sends JSON we construct ourselves; the consumer only reads from Firestore by Firebase UID.
- **Idempotent consumer** — the function tolerates duplicate deliveries (Pub/Sub guarantees at-least-once, not exactly-once).
- **Encrypted in transit** — all Pub/Sub traffic is TLS-encrypted within Google's network; messages are encrypted at rest with Google-managed keys (AES-256).

- **IAM-isolated** — only the `tradeson-api` Cloud Run service identity has `roles/pubsub.publisher`; only the Cloud Function's identity has `roles/pubsub.subscriber`. No external party can publish or consume.

---

4. Database Schema (Postgres)

Total: **30 tables** across six logical domains. Schema lives in `api/src/schema/migration.sql`. Every connection to the DB uses the Cloud SQL Auth Proxy with IAM authentication — no raw passwords in environment variables; no public IP on the instance.

4.1 Identity & Profile (12 tables)

Table	Purpose
`users`	Master user row. `firebase_uid` ties to Firebase Auth. `role` ∈ {homeowner, property_manager, realtor, licensed_tradesperson, unlicensed_tradesperson, admin}.
`user_addresses`	Mailing/billing addresses.
`user_notification_preferences`	SMS / email / push toggles.
`homeowner_profiles`	Property address, type, service interests.
`property_manager_profiles`	Company, plan type.
`managed_properties`	PM portfolio (one PM → many properties).
`realtor_profiles`	Brokerage, license number, service radius, referral code.
`realtor_clients`	Email-based client invitations.
`realtor_favorites`	Tradespeople a realtor marks for re-hire.
`tradesperson_profiles`	Business info, primary trades, sub-services, ID/insurance flags, Stripe Connect status, rating, jobs_completed, **compliance_status**.
`service_areas`	Zip codes a tradesperson serves.
`compliance_documents`	Tradesperson licenses + expiration + verification status.
`payout_accounts`	Stripe Connect Express account info.

4.2 Job lifecycle (6 tables)

Table	Purpose
`jobs`	Core job record. Joins homeowner → tradesperson. Carries status, category, severity, intake_answers (JSONB), AI summary, location.
`job_photos`	Photos attached to a job (intake / before / after / completion).
`quotes`	

Table	Purpose
	Tradesperson bids: price, hours, hourly_overage_rate, status, tool_inventory (JSONB).
appointments	Scheduled visit. References both job + quote.
appointment_checklist	Itemized job tasks the tradesperson checks off.
scope_changes	Tradesperson-initiated scope changes mid-job (with price delta + customer approval).

4.3 Payments (3 tables)

Table	Purpose
payments	One row per money movement. amount, platform_fee (10%), net_payout, Stripe IDs, status.
invoices	Customer-facing invoice (subtotal + tax + total + PDF URL).
invoice_line_items	Itemized invoice lines.

No card data is stored. All card details are tokenized by Stripe and held in Stripe's vault. We persist only Stripe object IDs.

4.4 Communication (3 tables)

Table	Purpose
conversations	Metadata for a messaging thread (participants, last_message_at). Messages themselves live in Firestore.
notifications	Per-user notification log (channel, delivered, read).
device_tokens	FCM tokens for push notifications.

4.5 Reviews

Table	Purpose
reviews	1–5 star + comment. UNIQUE per (job_id, reviewer_id) — a job cannot be reviewed twice by the same person.

4.6 Admin & Audit (4 tables)

Table	Purpose
flagged_accounts	Open admin queue (severity high/medium/low).
admin_resolutions	Admin actions: warning / suspension / deactivation / explanation_request.
audit_log	Append-only record of every admin action and key server-side mutation.

Table	Purpose
`match_events`	Matching telemetry — every job-board view, quote action, and outcome.

5. CRUD Event Catalog

Every user-triggered action that touches the backend. Use this when something feels off: open DevTools → Network → filter `api/v1`, do the action, compare the requests you see to the row below.

All routes prefixed with `${VITE_API_URL}/api/v1`. Production base: `https://tradeson-api-63629008205.us-central1.run.app`. Authenticated routes require `Authorization: Bearer <Firebase ID token>`.

5.1 Authentication

ID	Action	HTTP	Backend writes
E1	Sign up	POST `/users` 201	`users` row + default notification prefs
E2	Sign in	GET `/users/me` 200/404	none (read)
E3	Sign out	(client-side only)	none
E4	Self-heal (auto)	POST `/users` 201 + GET `/users/me` 200	`users` row inserted from Firebase UID
E5	Update profile	PUT `/users/me` 200	`users` row updated
E6	Delete account (soft)	DELETE `/users/me` 200	`users.is_active = false` (row preserved for audit)

5.2 Onboarding

ID	Role	Route	Writes
E7	Homeowner	POST `/onboarding/homeowner`	`homeowner_profiles` + `user_addresses` + notif prefs
E8	Licensed tradesperson	POST `/onboarding/licensed-trade`	`tradesperson_profiles` + `service_areas` + (optional) `compliance_documents`. Wrapped in PG transaction.
E9	Unlicensed tradesperson	POST `/onboarding/non-licensed-trade`	Same as E8 minus compliance.
E10	Property manager	POST `/onboarding/property-manager`	`property_manager_profiles` + addresses
E11	Realtor		

ID	Role	Route	Writes
		POST /onboarding/realtor	realtor_profiles + (optional) realtor_clients

5.3 Jobs

ID	Action	HTTP	Side effects
E12	Post a job	POST /jobs 201	jobs row + audit log + Pub/Sub job.created
E13	List jobs (auto-filtered by role)	GET /jobs 200	match_events shown rows
E14	View job detail	GET /jobs/:id 200	none
E15	Mark complete (tradesperson)	PATCH /jobs/:id/status 200	jobs.status = 'completed', auto-fires Stripe platform-payout, new payments row, Pub/Sub job.status_changed

5.4 Quotes

ID	Action	HTTP	Side effects
E16	Submit a quote	POST /quotes/:jobId/quotes 201	quotes row + audit + Pub/Sub quote.submitted
E17	Accept a quote	POST /quotes/:id/accept 200 + /stripe/authorize-job 200	Selected quote → accepted, siblings → rejected, jobs.assigned_tradesperson_id set, payments pre-auth hold row, audit, Pub/Sub quote.accepted

5.5 Payments

ID	Action	HTTP	Notes
E18	Payment history	GET /payments/me 200	Read-only
E19	Stripe Connect setup (tradesperson)	POST /stripe/create-connect-account 200	Opens Stripe-hosted onboarding popup
E20	Save payment method (customer)	POST /stripe/create-setup-intent 200	Card details collected by Stripe SDK; never touch our servers
E21	Pre-auth hold	(auto on E17)	New pending payments row
E22		(auto on E15)	

ID	Action	HTTP	Notes
	Auto-payout on completion		Stripe transfer minus 10% platform fee

5.6 Reviews

ID	Action	HTTP
E23	Submit review	POST `/reviews` 201
E24	List a tradesperson's reviews	GET `/reviews/:tradespersonId` 200

5.7 Realtor / Broker

ID	Action	HTTP
E25	Open realtor dashboard	GET `/realtor/dashboard` 200
E26	Favorite a tradesperson	POST `/realtor/favorites` 201
E27	Unfavorite	DELETE `/realtor/favorites/:id` 200

5.8 Admin (requires `admin: true` Firebase custom claim)

ID	Action	HTTP
E28	View compliance queue	GET `/admin/compliance` 200
E29	Decide on a submission	POST `/admin/compliance/:id/decision` 200
E30	Apply resolution to flagged account	POST `/admin/resolutions` 200
E31	Platform metrics	GET `/admin/metrics` 200

5.9 Stripe webhooks (server-to-server)

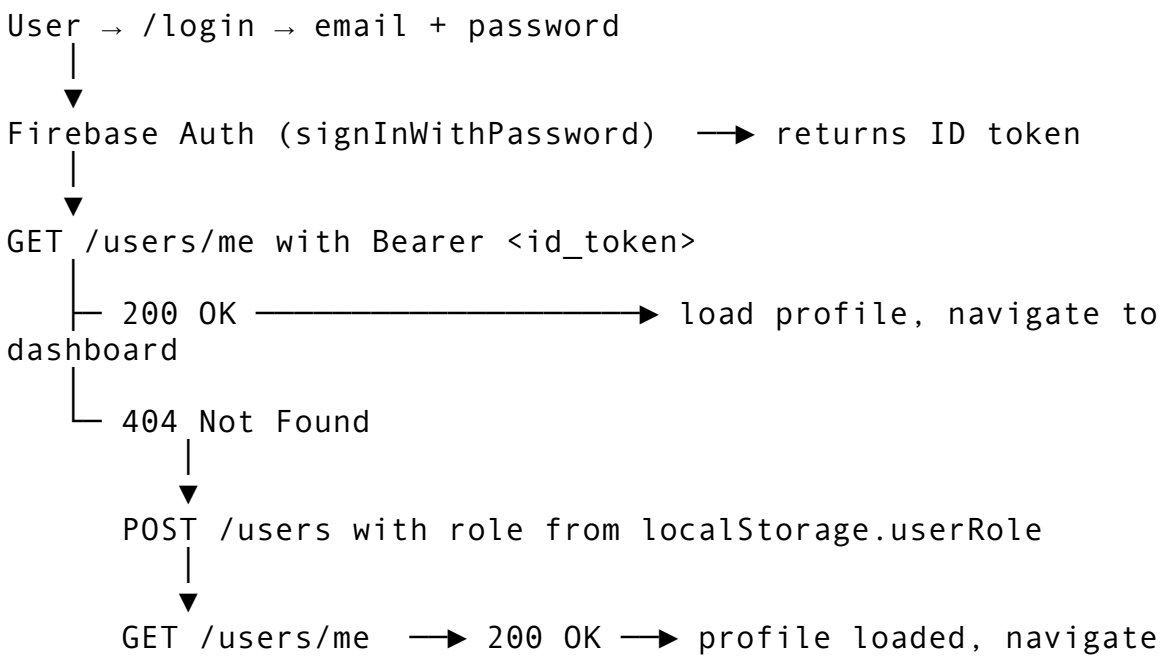
ID	Trigger	Notes
E32	Stripe webhook delivery	POST `/stripe/webhooks` — payment_intent.succeeded, account.updated, transfer.created, charge.dispute.created. Signature-verified with the Stripe webhook secret.

---

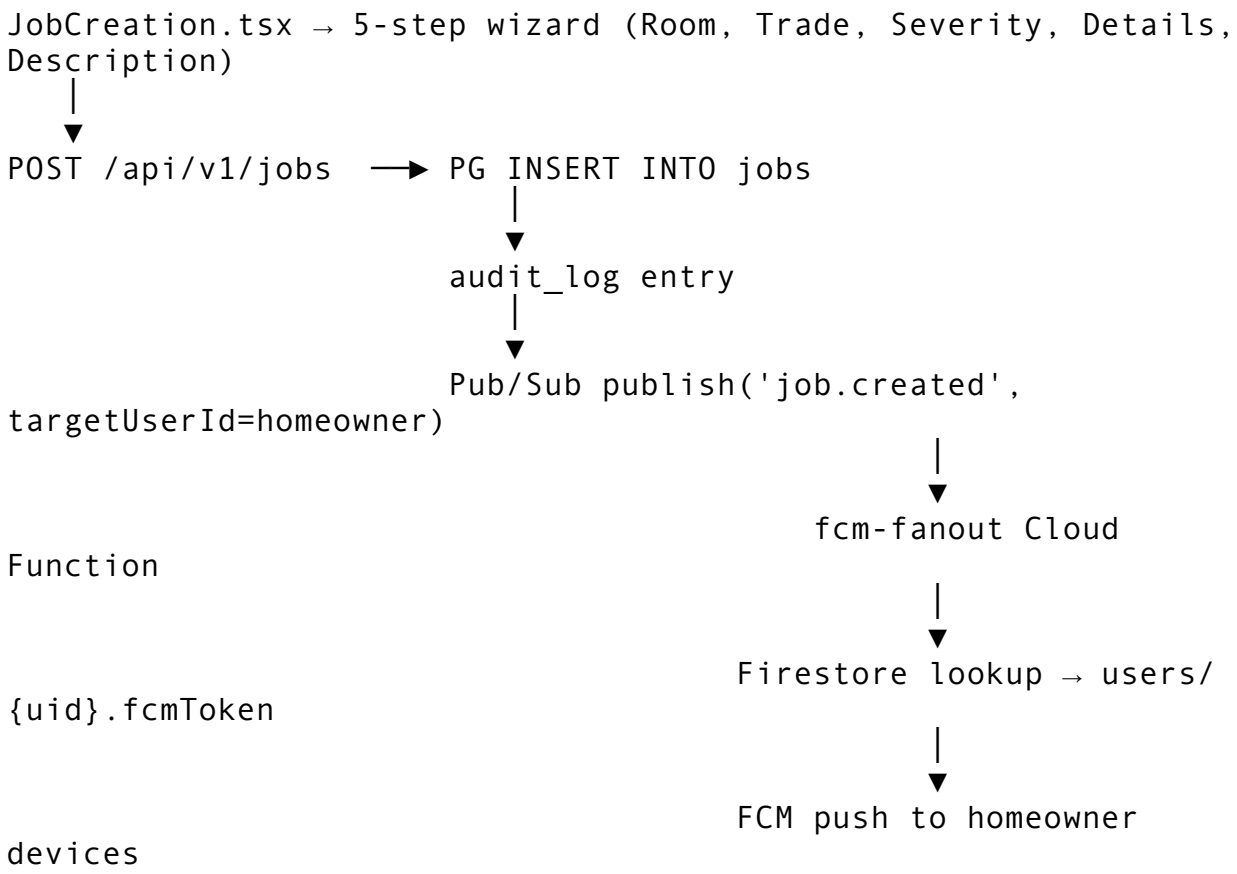
6. Key Process Flows

6.1 Login self-heal

The self-heal pattern (commit `760d004`) prevents users from being locked out if their Postgres row is missing despite a valid Firebase Auth account.

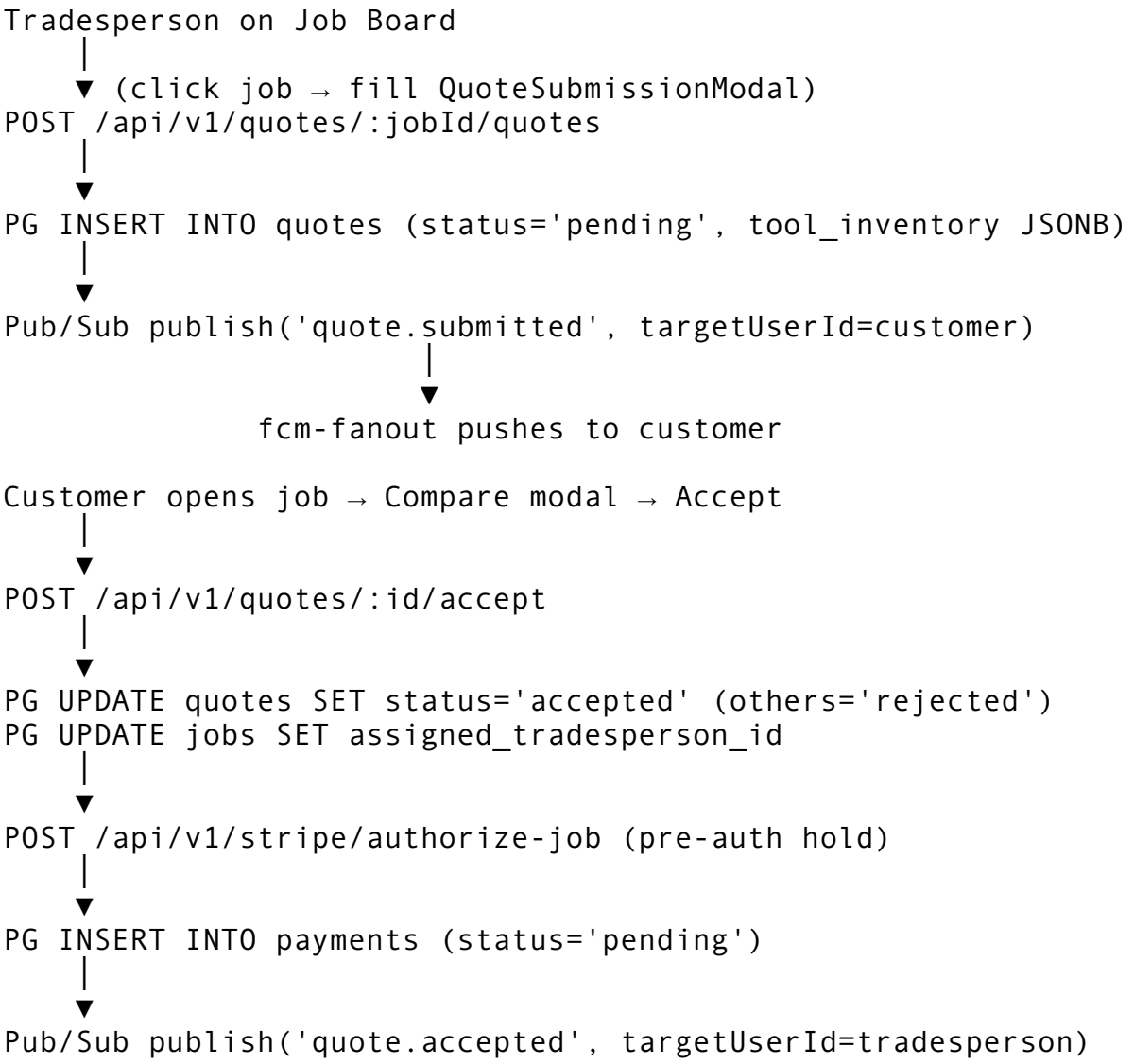


6.2 Job posting

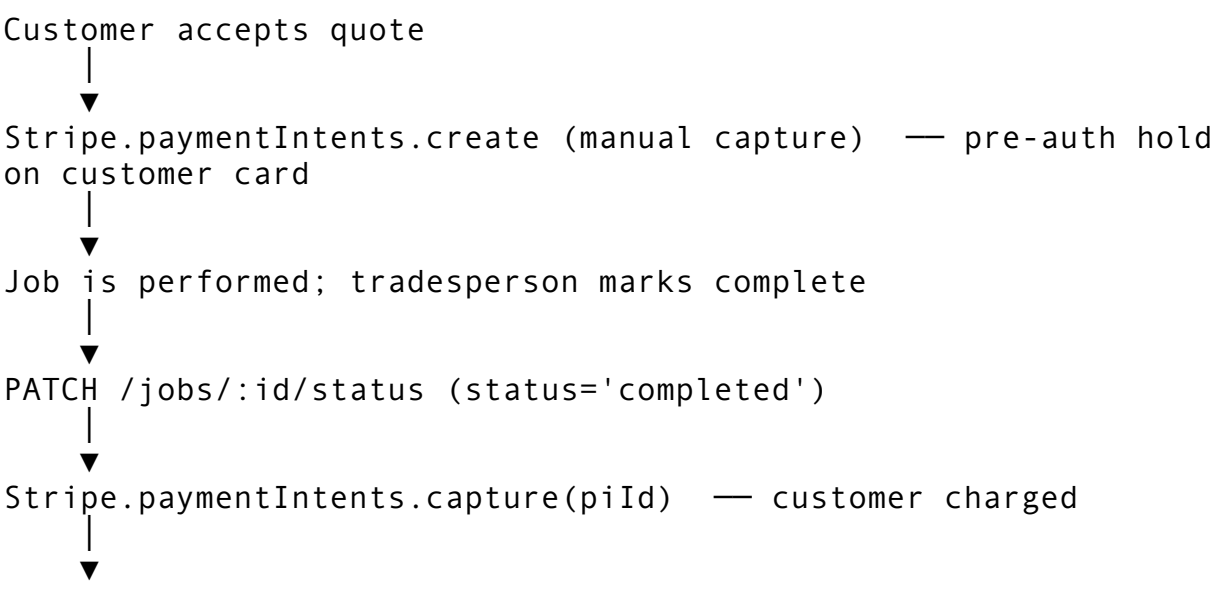


Photos are uploaded directly from the browser to **Firebase Storage** using a signed upload URL; only the resulting public URL is sent to the API. This keeps large binary uploads off our Cloud Run service entirely.

6.3 Quote submission and acceptance



6.4 Payment lifecycle



```
Stripe.transfers.create
   amount = total - 10% platform fee
   destination = tradesperson's Stripe Connect account
    │
    ▼
PG payments row → status='completed'
```

If the customer doesn't manually confirm within 3 hours of the tradesperson marking complete, a Cloud Scheduler cron job (`/internal/release-expired-holds` every 30 min) auto-captures the hold to release payment.

---

7. Security Posture

This chapter answers each section of the **Tokio Marine HCC TechGuard Cyber Liability Application (TEO-NBA 2.2021)**. Numbering matches the application form so the underwriter can cross-reference directly.

7.1 Section 4 — Records (data inventory)

Question	Answer
4.a Do you collect/store/process private or sensitive information?	**Yes.** ~40 unique electronic records as of May 2026 (early-stage / pre-launch). Categories: full name, email, phone number, mailing address, optional government-issued ID image (tradespeople only, stored in Firebase Storage with restricted access), insurance certificate image (tradespeople only), trade-license number (tradespeople only). **No** SSN. **No** date of birth. **No** healthcare records. **No** authentication passwords stored — Firebase Auth holds those.
4.b Biometric data?	**No.** No fingerprints, voiceprints, facial recognition, or any biometric identifiers collected or processed.
4.c Do you process credit card transactions?	**No.** All payments are processed through **Stripe Connect Express**. Card details are entered into Stripe-hosted UI (`<PaymentElement>` SDK rendered in an iframe owned by stripe.com). Card numbers, CVV, and expiration dates **never touch TradesOn servers**. Only opaque Stripe object IDs (`pi_…`, `pm_…`, `acct_…`) are stored. PCI-DSS scope is reduced to **SAQ-A** (eligible — merchant outsources all cardholder-data functions to a PCI-DSS validated third-party processor).

7.2 Section 5 — IT Department

Question	Answer
5.a Who is responsible for network security?	**Skylar McKeith Magaziner**, Co-Founder. Email: skylarmckeith@gmail.com. Phone: 605-430-2693.
5.b Outsourced or in-house?	**In-house** (managed by the founding team).
5.c IT personnel	4 total (2 co-founders + 2 CTOs).
5.d Dedicated security personnel	0 dedicated — security is a shared responsibility within the engineering team given the early stage of the company.

7.3 Section 6 — Information & Network Security Controls

6.a Cloud provider

Yes. Primary cloud provider: **Google Cloud Platform** (project ID `tradeson-491518`, billing project `frankly-data`, region `us-central1`). All compute, storage, database, and message-broker services are provisioned within GCP. No multi-cloud deployment; no on-premise infrastructure.

6.b MFA on cloud provider services

Yes (in process of formal verification). Co-founder Google accounts that hold Project Owner / Editor IAM roles on the GCP project should have **2-Step Verification** enforced at the Google account level (the standard Google Workspace setting). MFA factors: Google Prompt + recovery code. *Action item: confirm this is enabled on every account with* `roles/owner` *or* `roles/editor` *on the* `tradeson-491518` *project before submission.*

6.c Encryption of sensitive and confidential information

Yes — encrypted at rest and in transit, by default, via Google-managed keys.

Data store	At rest	In transit
Cloud SQL (Postgres)	AES-256, Google-managed encryption keys (Cloud KMS-backed)	TLS 1.2+ enforced; Cloud SQL Auth Proxy with IAM authentication
Firestore	AES-256, Google-managed	TLS 1.2+
Firebase Storage	AES-256, Google-managed	TLS 1.2+ via signed URLs
Cloud Pub/Sub	AES-256, Google-managed	TLS 1.2+
Firebase Auth tokens	Encrypted JWTs (RS256)	TLS-only delivery; short-lived (1 hour); refreshed via secure refresh tokens

All public-facing endpoints (frontend + API) are served over HTTPS with HSTS-equivalent headers via Cloud Run's managed TLS termination. Cloud Run automatically rotates TLS certificates.

If 6.c were "No," the compensating controls would be: - **(1) Server segregation**: Cloud SQL is on a private IP only. The application servers (Cloud Run) reach it via Cloud SQL Auth Proxy.

Direct access from the public internet is blocked at the network level. **Yes, in place.** - **(2) Role-based access control**: GCP IAM enforces least-privilege roles on the project. Application code uses `requireAuth + requireAdmin` middleware to gate admin routes. Firestore Security Rules enforce per-document read/write scopes (e.g., users can only read their own profile; admin queue collections are admin-only). **Yes, in place.**

7.4 Section 7 — Ransomware Controls

7.a Email pre-screening + sandbox detonation

- **Pre-screening: Yes** — Google Workspace Gmail provides built-in spam, phishing, and malicious attachment detection (Google Safe Browsing + ML-based classifier).
- **Automated sandbox detonation: No** — we do not currently run a dedicated email sandbox product (e.g., Proofpoint TAP, Mimecast). Given the team size (4) and the absence of business-critical wire-transfer workflows, this is a documented gap rather than an active risk.

7.b Remote access to your network

No traditional corporate network exists. TradesOn has no on-premise data center, no VPN, no RDP-accessible servers. Engineers access GCP and source code via:

- **GitHub** over HTTPS with personal access tokens / SSH keys; MFA required on the GitHub organization.
- **Google Cloud Console** over HTTPS, MFA required on the Google account.
- **gcloud CLI** with Application Default Credentials tied to the developer's MFA-protected Google account.

There is no RDP, no SSH-into-production. Production deployments happen via push-to-branch CI (Cloud Build).

7.c Web application MFA for end-users

- **Customers and tradespeople** authenticate via Firebase Auth with email + password. **MFA is not currently enforced on end-user accounts** — this is consistent with consumer SaaS norms (Uber, DoorDash, Angi) but is on the product roadmap (Firebase Auth supports SMS MFA via the same SDK).
- **Engineering team and admin users** must use MFA-protected Google accounts to reach the GCP project, source code, and the admin dashboard.

7.d NGAV (next-generation antivirus)

Endpoints are MacBooks owned by founding team members, protected by:

- **Apple XProtect** (macOS built-in malware scanner; updated daily).
- **Apple Gatekeeper** (code-signing enforcement on all executables).
- **System Integrity Protection (SIP)** enabled.
- **FileVault disk encryption** enabled.

We do not run a commercial NGAV product (CrowdStrike, SentinelOne, Carbon Black) on developer endpoints. Documented gap; revisit as headcount grows.

7.e EDR (endpoint detection and response)

Same posture as 7.d — no commercial EDR deployed on developer endpoints. macOS unified logging is available locally but is not centrally aggregated.

7.f MFA on privileged user accounts

Yes. All privileged accounts that can modify production infrastructure (GCP IAM Owner/Editor, GitHub org admin, Stripe Dashboard owner, Firebase project owner) are protected by **MFA at the identity-provider level** (Google account 2-Step Verification, GitHub MFA, Stripe MFA). No shared service account passwords; no static API keys committed to source control.

7.g Backups

Question	Answer
Frequency	**Daily** for Cloud SQL (automated point-in-time recovery enabled by default for the past 7 days).
Restore time for essential functions	**0–24 hours** (Cloud SQL clone-from-backup typically completes within 1–2 hours for a database of this size).
Backups encrypted	**Yes** — Cloud SQL backups encrypted at rest with the same Google-managed key as the live instance.
Off-network / air-gapped	**Yes** — Cloud SQL backups are stored in a Google-managed bucket isolated from the production database; not accessible from application code.
Different credentials than live	**Yes** — restore requires GCP IAM permission `cloudsql.backupRuns.use`, distinct from runtime DB credentials.
MFA on backup access	**Yes** — implicit, since the GCP console requires MFA on the Owner account.
Cloud-syncing service for backups	**No** (we don't use Dropbox/OneDrive/Drive for backups; Cloud SQL native backups only).
Tested restore in last 6 months	**Action item: schedule a documented restore drill before submission.**
Integrity tested before restore	Cloud SQL validates checksums on restore; we do not currently run separate malware scans on backup snapshots.

7.5 Section 8 — Phishing Controls

8.a Social engineering training

- Employees with financial responsibilities: **No formal program yet** (one founder handles all financial decisions; no separate accounting team).
- Phishing simulation: **No** (deferred until headcount > 10).

The team is small enough that any unusual financial request triggers a verbal confirmation between co-founders.

8.b Wire transfer controls

Not applicable / out of scope. TradesOn does **not** send or receive wire transfers as part of normal operations. All inbound payment from customers and outbound payouts to tradespeople flow through **Stripe Connect**. Stripe's own controls (KYC, account verification, 2FA on transfers) apply to all money movement. The platform itself never initiates ACH or wire transfers between our bank accounts and external parties as part of automated workflows.

7.6 Sections 12–13 — Loss History

Question	Answer
Past 3 years: any complaints, demands, litigation re: professional E&O?	**No.** Company founded March 2026; no claims to date.
Past 3 years: any privacy / network security / breach claims?	**No.**
Past 3 years: government investigation re: privacy law?	**No.**
Notified anyone of a security or privacy breach?	**No.**
Cyber extortion demand or threat?	**No.**
Unscheduled network outage or interruption?	**No.**
Property damage or BI loss from cyber-attack?	**No.**
Wire transfer fraud / phishing fraud loss?	**No.**
Knowledge of any wrongful act that may give rise to a claim?	**No.**

8. Application-Level Security Controls (additional detail)

This section goes beyond the form questions and documents the controls we want the underwriter to know about.

8.1 Authentication

- **Identity provider**: Firebase Authentication (a Google-managed service). Passwords are never stored by TradesOn — Firebase salts and hashes them with `scrypt`.
- **Token format**: Every authenticated API call carries a Firebase-issued **RS256-signed JWT** (ID token) in the `Authorization: Bearer …` header.
- **Token lifetime**: 1 hour. Refresh tokens are stored in `localStorage` with the `__FIREBASE_DEFAULTS__` key; the SDK rotates the ID token automatically.
- **Custom claims**: Admin users carry a `admin: true` custom claim (set out-of-band via the Firebase Admin SDK by an existing admin). The API enforces admin routes via `requireAdmin` middleware that cryptographically verifies the claim on every request.

8.2 Authorization

- **Role-based middleware** in Express (`api/src/middleware/auth.ts`):
  - `requireAuth` — verifies the Firebase ID token signature and freshness on every request.
  - `requireAdmin` — additionally requires the `admin: true` custom claim.
- **Per-user data scoping** in routes — every list/detail query is bound to the caller's UID. Customers see only their own jobs; tradespeople see only open jobs in their service area + categories.
- **Firestore Security Rules** (deployed in `firestore.rules`):
  - `threads/{threadId}` and `threads/{id}/messages/{msgId}` — read/write only for thread participants.
  - `support_tickets/{id}` — create by any auth'd user; read/update by admin only; never delete.
  - `audit_log/{id}` — create by auth'd user; read by admin only.
  - **Default deny** on all other paths.

8.3 Input validation and injection prevention

- **SQL injection**: All Postgres queries use **parameterized statements** via the `pg` driver (`pool.query(text, [params])`). No string concatenation into SQL ever.
- **NoSQL injection**: Firestore queries use the typed SDK (`collection().where('field', '==', value)`) — no eval-style query construction.
- **XSS**: React 18 escapes all interpolated values by default; we do not use `dangerouslySetInnerHTML` anywhere.
- **CORS**: Locked to known origins (the production frontend domain + localhost during development).
- **Rate limiting**: Not currently enforced at the application level. Cloud Run autoscales but charges per request; planned to add `express-rate-limit` per IP before public launch.

8.4 Secret management

- All secrets (Stripe API keys, Firebase service account JSON, internal cron secret, database credentials) are injected as **Cloud Run environment variables** — never committed to source control.
- The Stripe webhook secret is verified on every inbound webhook via `stripe.webhooks.constructEvent`.
- The internal cron endpoint (`/internal/release-expired-holds`) requires a shared `INTERNAL_SECRET` header that matches a Cloud Run env var; only Cloud Scheduler holds this secret.

8.5 Audit logging

- The `audit_log` Postgres table records every admin action and every key user mutation (job created, quote accepted, payment captured, compliance decision).
- Cloud Run access logs and Cloud Audit Logs (GCP-native) record every API call, every IAM change, and every database connection. Retention: 30 days standard / 400 days for admin activity (Google default).
- Stripe Dashboard maintains an immutable log of every payment, transfer, and dispute.

8.6 Vulnerability and patch management

- **Frontend dependencies** are scanned via `npm audit` on every push (Cloud Build runs `npm install` which surfaces high-severity advisories).
- **Backend dependencies** use a **Husky pre-push git hook** (`.husky/pre-push`, installed 2026-05-05) that runs `tsc` against both packages and blocks the push on any TypeScript error. This prevents the silent-build-failure pattern that affected the API repo from April–May 2026.
- **GCP base images**: Cloud Run uses Google-maintained Node.js 20 and nginx images, automatically patched.
- **Disclosure policy**: We do not currently publish a `security.txt` or formal bug-bounty program (planned post-launch).

8.7 Incident response

- **Detection**: Cloud Run logs alert on error-rate spikes (planned: Cloud Monitoring alert policy at >1% 5xx rate). Firestore and Cloud SQL surface anomalous query patterns in the GCP console.
- **Containment**: We can revoke a compromised Firebase user's access in seconds via the Firebase console (force sign-out + delete user). We can revoke a compromised IAM identity in seconds via GCP IAM. We can rotate the Stripe API key in minutes via the Stripe dashboard.
- **Communication**: A breach affecting customer PII would be reported to affected users within 72 hours per industry norm (CCPA / GDPR-aligned timeline), even though current user volume is below most regulatory thresholds.
- **Recovery**: Cloud SQL point-in-time restore lets us rewind the database to any second within the last 7 days.

8.8 Third-party data processors

Vendor	Data accessed	Compliance posture
Google Cloud Platform	All transactional data, files, messaging	SOC 2 Type II, ISO 27001/27017/27018, HIPAA-eligible
Firebase (Google)	Authentication identities, push tokens, messaging	Same as GCP
Stripe	Cardholder data, KYC for tradespeople (Connect)	PCI-DSS Level 1 service provider, SOC 2 Type II
GitHub (Microsoft)	Source code	SOC 2 Type II, ISO 27001

We do not share user data with marketing platforms, ad networks, or data brokers.

9. Operational Reliability

Concern	Control
Single region (us-central1)	All services pinned to `us-central1` for latency and cost. Cloud SQL has automated daily backups; Firestore has automatic multi-region replication within Google's network.

Concern	Control
Cold starts	Cloud Run is configured to scale to zero today (cost optimization). Min-instances = 1 is on the launch checklist.
Dependency on Stripe	If Stripe is down, money movement stalls but the app remains usable for browsing and messaging. Stripe webhook retries handle transient delivery failures (up to 3 days).
Dependency on Firebase Auth	If Firebase Auth is down, no new logins succeed. Existing logged-in sessions continue working until ID token expiry (1 hour).
Build pipeline integrity	Husky pre-push hook + Cloud Build trigger gating ensures broken commits never reach production.
Deployment	Automated via push to `production` branch → Cloud Build → Cloud Run. Rollback is a single-click revert in the Cloud Run console (previous revision is always retained).

---

10. Data Subject Rights

In the event a user requests access, correction, or deletion of their personal data:

- **Access**: Full data export available on request — joins all tables on `user_id` and produces a JSON file.
- **Correction**: User can self-edit name, phone, email via `/profile`; admin can update any field via the admin dashboard.
- **Deletion**: User-initiated soft delete via `/privacy-settings` flips `users.is_active = false`. Hard deletion (full row + cascading PG deletes + Firestore collection sweep + Firebase Auth user removal) is performed manually on request — typically within 30 days.

---

11. Document Maintenance

Field	Value
Document version	1.0
Generated	2026-05-10
Owner	TradesOn Engineering
Sources	`api/src/schema/migration.sql`, `api/src/routes/*`, `firestore.rules`, `functions/fcm-fanout/index.js`, `scripts/testPubsub.mjs`, GCP project `tradeson-491518`
Reviewer for cyber insurance submission	Skylar McKeith Magaziner, Co-Founder
Next review	Before any new third-party data processor is onboarded, or annually, whichever is sooner

Appendix A — Verified Pub/Sub Test Run (2026-05-10)

```
$ node scripts/testPubsub.mjs
auth: Application Default Credentials

Publishing 4 test events to tradeson-491518/topics/tradeson-
events

  ✓ job.created            messageId=19012479423372511
  ✓ quote.submitted        messageId=19012274884235178
  ✓ quote.accepted         messageId=19012236212861714
  ✓ job.status_changed     messageId=19012447211756537

Result: 4 published, 0 failed.

$ gcloud functions logs read fcm-fanout --region us-central1 --
limit 10

[job.status_changed]    no fcmToken for user TEST-UID — skipping
push
[quote.accepted]        no fcmToken for user TEST-UID — skipping
push
[quote.submitted]       no fcmToken for user TEST-UID — skipping
push
[job.created]           no fcmToken for user TEST-UID — skipping
push
```

All 4 events published from the local test script were consumed by `fcm-fanout` within ~1 second. The "no fcmToken" message is the expected no-op path when a sentinel UID is used; with a real Firebase UID whose token is registered, the function would call `admin.messaging().send()` and a push notification would be delivered to the user's device.

Appendix B — Repository Layout

```
tradeson/
├── src/                    React 18 + TypeScript frontend
│   ├── pages/              Screen components (one per page)
│   ├── components/         Reusable UI
│   ├── services/
│   │   ├── api.ts          Cloud Run API client
│   │   ├── firebase.ts     Firebase init (auth + Firestore +
FCM)
│   │   └── messagingService.ts
│   └── contexts/AuthContext.tsx
│
├── api/                    Cloud Run service (Node.js + Express)
│   ├── src/
```

```
│   │   ├── index.ts        Express bootstrap, middleware, route
registration
│   │   ├── middleware/auth.ts   requireAuth + requireAdmin
│   │   ├── routes/         users, jobs, quotes, payments,
stripe, admin, …
│   │   ├── services/pubsub.ts   Pub/Sub publisher
│   │   └── schema/migration.sql Postgres DDL
│   └── package.json
│
├── functions/fcm-fanout/   Cloud Function (gen2) — Pub/Sub → FCM
│   ├── index.js
│   └── package.json
│
├── scripts/
│   ├── testPubsub.mjs      End-to-end Pub/Sub test
│   └── setAdminClaim.mjs   Set Firebase admin custom claim
│
├── docs/backend/
│   ├── ERD.md              Mermaid ERD + table descriptions
│   ├── DATA_MAPPING.md     Table → routes → frontend consumers
│   ├── EVENTS.md           CRUD event catalog (deeper than §5)
│   └── TECH_GUIDE.md       This document
│
├── firestore.rules         Firestore security rules
├── .husky/pre-push         Pre-push gate: builds both packages
├── cloudbuild.yaml         Cloud Build pipeline
├── Dockerfile              Multi-stage build
└── nginx.conf              Cache headers
```

End of document. For questions about specific implementation details, contact the engineering team. For underwriter follow-up on any answer in §7, contact Skylar McKeith Magaziner.